ComicCraft – Develop the Core Functionalities

Development of Core AI and Backend Functions

Project: ComicCraft – AI Comic Story Creator
Technology: FastAPI, Jinja2, Gemini models, Stable Diffusion, FPDF
Topic covered: Core functionalities and FastAPI integration

This assignment is organized as a standalone technical note with clear concepts, implementation points, workflow diagrams, and practical checkpoints.

Prepared for Academic Submission

1. Core Functionalities – Overview

The core functionality of ComicCraft is the automated conversion of a creative prompt into a structured comic. The main functions are separated into modules so that each part can be tested and improved independently.

generate_outline()	generate_story()	generate_image()	build_comic_layout()	save_pdf()

Main Functions

Function	Module	Main Output
generate_outline()	gemini_flash.py	Structured five-panel outline.
generate_story()	gemini_pro.py	Narration and character dialogue.
generate_image()	image_generator.py	Saved comic-style image path.
build_comic_layout()	layout_builder.py	Panel list containing image and text data.
save_pdf()	exporters.py	Final multi-page PDF file.

Design Goal

The functions should form a predictable pipeline. Each function receives clear input and returns clear output. This modular design improves readability, debugging, testing and future upgrades.

2. Generating the Structured Comic Outline

The first major functionality is outline generation. The generate_outline() function uses a Gemini model to transform the user's idea into a structured five-panel plan. The outline acts as the blueprint for the remaining stages.

Expected Panel Information

- **Panel number:** identifies the sequence.
- **Panel title:** gives a short description of the scene.
- **Scene description:** explains what should happen visually and narratively.
- **Image prompt:** provides instructions for the image-generation model.

Example Structure

```
[ {"panel": 1, "title": "The Discovery", "scene": "A fox enters a glowing forest.",
"image_prompt": "cartoon fox in enchanted forest"}, {"panel": 2, "title": "The Hidden
Path", "scene": "The fox follows a mysterious trail.", "image_prompt": "forest path with
magical lights"} ]
```

Benefits

- Creates a consistent sequence before detailed writing begins.
- Provides image prompts that are connected to the story.
- Makes the number and order of comic panels predictable.
- Allows the application to validate the data before continuing.

Extra Point: Prompt Quality

A strong outline prompt can request consistent characters, clear scenes, concise descriptions and separate image-generation prompts. This reduces ambiguity between the text and visual stages.

3. Story, Image and Layout Generation

A. generate_story()

After the outline is ready, generate_story() expands the panel descriptions into a complete comic narrative. It can produce captions, narration and character dialogue while preserving the order of the panels.

B. generate_image()

The image-generation function sends each panel's image prompt to the selected image model. The generated image is saved in a dedicated static/panels directory, and the file path is returned to the application.

C. build_comic_layout()

The layout builder connects the generated images with the corresponding panel text. The result is a structured list containing the panel number, image path and narrative content. This becomes the common data source for preview and PDF export.

Outline	Narration	Image Files	Layout List

Why Separation Matters

- Changing the image model does not require rewriting the story generator.
- The layout can be redesigned without changing the AI prompts.
- Individual modules can be tested with sample inputs.
- Failures can be located more easily because responsibilities are separated.

4. FastAPI Backend Integration

FastAPI connects the core functions to the web interface. The /generate route receives form data and orchestrates the complete pipeline. The /generate-comic/json route provides a JSON-based API alternative, while /test-image can be used to test image generation independently.

Route	Input	Purpose
/	Browser request	Loads index.html.
/generate	HTML form data	Runs the full comic generation pipeline and renders the preview.
/generate-comic/json	JSON body	Runs generation for API clients and returns structured data.
/test-image	Image prompt	Tests image generation without creating a complete comic.
/export-success	Export result	Shows successful export confirmation.

Full Backend Pipeline

Receive Input	Validate	Outline	Story	Images	Layout	PDF

Validation and Error Handling

- Pydantic can validate JSON request data.
- Form fields should be checked for required values.
- HTTPException can report controlled API failures.
- try-except blocks can handle unexpected model, parsing or file errors.
- The application should avoid returning incomplete output as if generation succeeded.

5. Exporting the Comic and Testing Core Functions

The save_pdf() function uses FPDF to combine the comic panels and narrative into a multi-page document. A successful export provides a practical endpoint for the entire generation pipeline.

Testing Strategy

Test	What to Verify	Expected Result
Outline test	Send a simple story prompt	Correct number and structure of panels.
Story test	Use a known outline	Narration and dialogue match panel order.
Image test	Use one image prompt	An image is generated and saved correctly.
Layout test	Use sample images and text	Correct pairing and ordering.
PDF test	Use a small layout	Readable PDF is created.
Integration test	Run /generate end to end	Complete comic is produced without manual intervention.

Conclusion

Developing the core functionalities as independent modules gives ComicCraft a clean and maintainable architecture. The central idea is simple: structured planning first, creative text second, visual generation third, layout fourth, and export last.

Key takeaway: Modular functions make the AI pipeline easier to test, debug, reuse and extend.

Reference: Core functionality details are based on the supplied ComicCraft project document.